

Normalização de Banco de Dados

Aprenda a organizar tabelas relacionais para reduzir redundância, evitar anomalias e garantir integridade dos dados — do conceito à prática com 1FN, 2FN e 3FN.

1FN

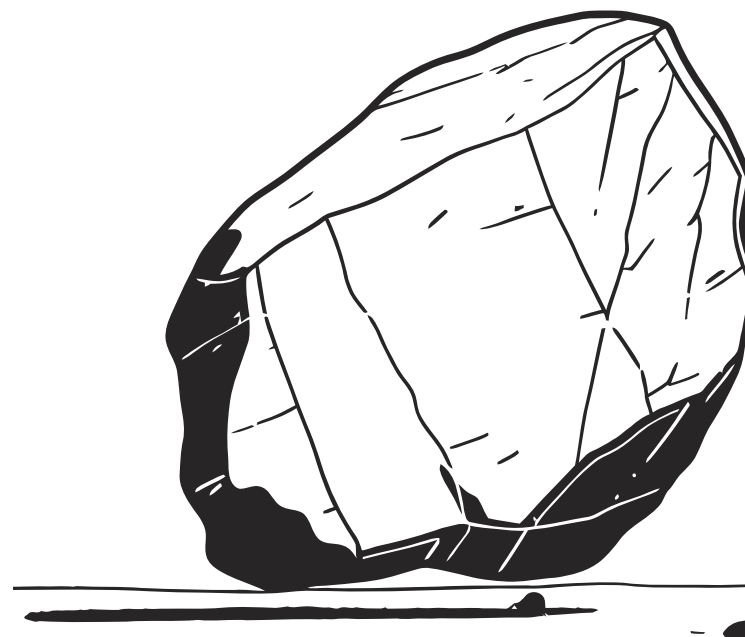
Valores atômicos

2FN

Dependência total da PK

3FN

Sem dependência transitiva



FUNDAMENTOS

O Que É Normalização e Por Que Normalizar

Normalização é o processo de organizar as tabelas de um banco relacional para **reduzir redundância**, evitar anomalias de atualização e garantir integridade. Foi formalizada por **E. F. Codd** a partir de 1970 com as Formas Normais (1FN, 2FN, 3FN...).



→ Elimina dados duplicados

Evita que a mesma informação apareça em várias linhas da tabela.

→ Evita anomalias

Previne problemas de inserção, atualização e exclusão inconsistentes.

→ Facilita manutenção

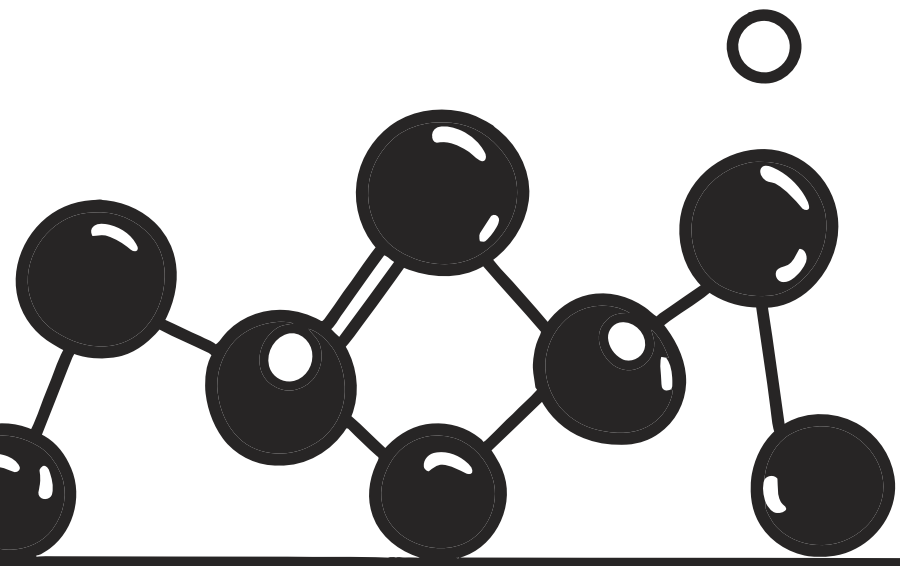
Torna o banco mais fácil de evoluir e manter ao longo do tempo.

→ Aumenta a integridade

Uma informação, um lugar — sem contradições entre registros.

→ 1FN, 2FN e 3FN resolvem 95% dos casos

As três primeiras formas normais cobrem a grande maioria dos cenários práticos.



1ª Forma Normal (1FN) — Atomicidade

Uma tabela está na **1FN** quando cada célula contém um único valor (atômico) e não existem grupos repetidos nem listas dentro de uma mesma coluna. É o **ponto de partida obrigatório** para qualquer normalização.



Valor único por célula

Cada coluna armazena UM valor por linha — atômico e indivisível.



Sem colunas multivaloradas

Proibido: `telefones: 9999, 8888` em uma única célula.



Sem grupos repetidos

Proibido: `curso1, curso2, curso3` como colunas separadas na mesma linha.



Chave primária obrigatória

Cada linha deve ser identificável de forma única por uma chave primária.

1FN — Exemplo Prático: Antes x Depois

Veja o problema clássico do campo multivalorado e como a 1FN o resolve dividindo a tabela em entidades separadas.

✖ ANTES — VIOLA 1FN

Tabela ALUNOS

id	nome	cursos
1	Ana	Java, SQL, Git
2	Bruno	Python, SQL

⚠ O campo **cursos** contém múltiplos valores separados por vírgula — viola a atomicidade da 1FN.

✔ DEPOIS — EM 1FN

Tabela ALUNOS

id	nome
1	Ana
2	Bruno

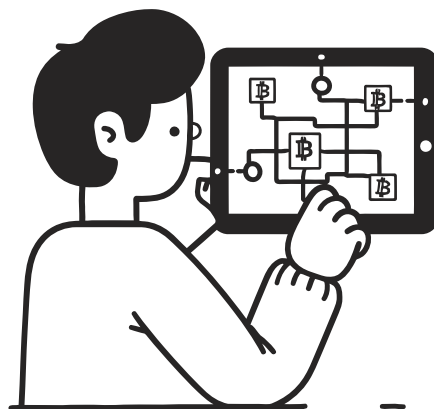
Tabela MATRICULAS

aluno_id	curso
1	Java
1	SQL
1	Git
2	Python
2	SQL

✔ Uma linha por curso — cada célula contém exatamente um valor atômico.

2FN — Dependência da Chave Inteira

Estando em 1FN, a tabela atinge a **2FN** quando todo atributo não-chave depende da chave primária **INTEIRA** — nunca apenas de parte dela. Só faz sentido em tabelas com chave primária composta.



Pré-requisito: 1FN

A tabela já deve estar na Primeira Forma Normal antes de aplicar a 2FN.

PK Composta

Aplica-se a tabelas com chave primária formada por duas ou mais colunas.

Dependência Total

Cada coluna não-chave deve depender de **TODA** a PK, não de parte dela.

Solução: Separar Tabelas

Colunas que dependem só de parte da PK devem ir para novas tabelas independentes.

2FN — Exemplo Prático: Antes x Depois

Quando a tabela MATRICULAS tem PK composta (aluno_id + curso_id), atributos do aluno ou do curso na mesma tabela violam a 2FN por dependerem apenas de parte da chave.

✗ ANTES — VIOLA 2FN

Tabela MATRICULAS

aluno_id	curso_id	nome_aluno	nome_curso	nota
1	10	Ana	Java	9.5
2	10	Bruno	Java	8.0

⚠ nome_aluno depende só de aluno_id.

⚠ nome_curso depende só de curso_id.

✓ DEPOIS — EM 2FN

ALUNOS

id	nome
1	Ana
2	Bruno

CURSOS

id	nome
10	Java

MATRICULAS

aluno_id	curso_id	nota
1	10	9.5
2	10	8.0

3ª Forma Normal (3FN) — Sem Dependência Transitiva

Estando em 2FN, a tabela atinge a **3FN** quando nenhum atributo não-chave depende de outro atributo não-chave. Em outras palavras: tudo depende **DA chave, DA chave INTEIRA, e SÓ da chave** (Codd).



Pré-requisito: 2FN

A tabela já deve estar na Segunda Forma Normal antes de aplicar a 3FN.



Sem Dependência Transitiva

Proibido: $A \rightarrow B \rightarrow C$, onde B e C são atributos não-chave. Exemplo: `funcionario_id` $\rightarrow$ `departamento_id` $\rightarrow$ `nome_departamento`.



Separar em Nova Tabela

Atributo que descreve outro atributo não-chave deve ser movido para uma tabela própria.



Um Conceito por Tabela

Resultado: cada tabela representa um único conceito coeso e bem definido.

 **Regra de Ouro (Codd):** "Todo atributo não-chave deve depender DA chave, DA chave INTEIRA, e SÓ da chave."

3FN — Exemplo Prático: Antes x Depois

O caso clássico: na tabela FUNCIONARIOS, o nome do departamento depende do departamento_id, não do funcionário. Isso é uma **dependência transitiva** que viola a 3FN.

✗ ANTES — VIOLA 3FN

Tabela FUNCIONARIOS

id	nome	dept_id	nome_dept	local_dept
1	Carlos	5	TI	Sala 3
2	Diana	5	TI	Sala 3

⚠ nome_dept e local_dept dependem de dept_id, não do id do funcionário.

Dependência transitiva: id → dept_id → nome_dept

✓ DEPOIS — EM 3FN

FUNCIONARIOS

id	nome	departamento_id
1	Carlos	5
2	Diana	5

DEPARTAMENTOS

id	nome	localizacao
5	TI	Sala 3

✓ FUNCIONARIOS.departamento_id → DEPARTAMENTOS.id

Cada tabela representa um único conceito coeso.

Comparativo Rápido: 1FN → 2FN → 3FN

Em uma única frase, a **regra de ouro** da normalização até 3FN é: *"todo atributo depende DA chave, DA chave inteira, e SÓ da chave"*.

1FN — Valores Atômicos

"DA chave"

- Cada linha tem chave primária
- Sem listas em células
- Sem grupos repetidos
- Cada célula = 1 valor

2FN — Dependência Total

"DA chave INTEIRA"

- Pré-requisito: 1FN
- Relevante em PK composta
- Sem dependência parcial
- Atributos dependem de toda a PK

3FN — Sem Transitiva

"SÓ da chave"

- Pré-requisito: 2FN
- Sem $A \rightarrow B \rightarrow C$
- Atributos não-chave independentes
- Uma tabela = um conceito

Regra de Ouro (Codd): "Todo atributo não-chave deve depender DA chave, DA chave INTEIRA, e SÓ da chave."

Normalize Esta Tabela

Aplique **1FN**, **2FN** e **3FN** passo a passo a uma tabela do mundo real. Trabalhe em duplas.
Tempo estimado: **25 minutos**.

Tabela PEDIDOS (não normalizada)

pedido_id	data	cliente_id	nome_cliente	cidade_cliente	produtos
101	2024-01-10	5	Fernanda	São Paulo	Mouse, Teclado, Monitor
102	2024-01-11	7	Ricardo	Curitiba	Notebook



Problemas identificados:

- produtos é uma string multivalorada (viola 1FN)
- nome_cliente e cidade_cliente dependem de cliente_id (viola 3FN)

O Que Entregar



Versão em 1FN

Resolva o campo produtos multivalorado separando em linhas individuais.



Versão em 2FN

Verifique se há dependências parciais após aplicar a 1FN e corrija se necessário.



Versão Final em 3FN

Apresente as tabelas resultantes com suas chaves primárias (PK) e estrangeiras (FK).



Entregue em papel ou diagrama digital ao final dos 25 minutos.